# Smart Tank Monitoring System

Relazione tecnica – Assignment 03

Embedded Systems and Internet of Things
Anno accademico 2025/2026

Docente: Alessandro Ricci


| **Autori** | **Matricole** |
|---|---|
| Lucas Prati | 0001117339 |
| Samuele D'Ambrosio | 0001125837 |


10 novembre 2026

# Indice


**1 Introduzione e obiettivi** **2**
- 1.1 Contesto e obiettivi del progetto . . . 2
- 1.2 Comportamento e requisiti principali . . . 2

**2 Architettura generale del sistema** **3**
- 2.1 Componenti e confini di responsabilità . . . 3
- 2.2 Comunicazione e propagazione dello stato . . . 3
- 2.3 Stato globale e disponibilità . . . 3
- 2.4 Scelte architetturali . . . 4

**3 Tank Monitoring Subsystem (TMS)** **5**
- 3.1 Architettura software . . . 5
- 3.2 Circuito e componenti hardware . . . 5
- 3.3 Task del firmware . . . 6
- 3.4 Test, criticità e taratura . . . 8

**4 Control Unit Subsystem (CUS)** **10**
- 4.1 Architettura software . . . 10
- 4.2 Gestione dello stato e politica automatica . . . 11
- 4.3 Comunicazione e sincronizzazione . . . 13
- 4.4 Validazione e limitazioni . . . 13

**5 Water Channel Subsystem (WCS)** **14**
- 5.1 Architettura e hardware . . . 14
- 5.2 Scheduler, task e macchine a stati . . . 15
- 5.3 Interfaccia operatore e controllo della valvola . . . 16
- 5.4 Validazione e limitazioni . . . 16

**6 Dashboard Subsystem (DBS)** **17**
- 6.1 Architettura dell'interfaccia web . . . 17
- 6.2 Comunicazione HTTP e gestione dello stato . . . 17
- 6.3 Interazione dell'operatore . . . 17

**7 Integrazione e comunicazione** **18**
- 7.1 Interazione tra sottosistemi e contratti . . . 18
- 7.2 Scenari operativi e gestione delle interruzioni . . . 19
- 7.3 Validazione del sistema integrato . . . 20

**8 Conclusioni** **21**
- 8.1 Risultati e limitazioni . . . 21
- 8.2 Possibili sviluppi futuri . . . 21

# 1 Introduzione e obiettivi

## 1.1 Contesto e obiettivi del progetto

Il progetto realizza un prototipo distribuito per monitorare il livello dell'acqua piovana in una tanica e controllare l'apertura di un canale di scarico. Il sistema prevede l'acquisizione periodica delle misure, il controllo automatico dell'apertura e la possibilità di intervento da parte dell'operatore, sia localmente sia attraverso un'interfaccia web.

La consegna dell'Assignment 03 definisce i requisiti funzionali, i sottosistemi e le modalità di comunicazione previste [1]. Il sistema è composto da quattro sottosistemi: il Tank Monitoring Subsystem (TMS), responsabile dell'acquisizione delle misure; il Control Unit Subsystem (CUS), che coordina il funzionamento del sistema; il Water Channel Subsystem (WCS), che gestisce l'attuatore e l'interfaccia locale; e il Dashboard Subsystem (DBS), che permette il monitoraggio e il controllo remoto.

## 1.2 Comportamento e requisiti principali

Il sistema si avvia in modalità `AUTOMATIC`, nella quale l'apertura del canale viene regolata in base al livello dell'acqua e alle soglie temporali previste. In modalità `MANUAL`, l'operatore può impostare direttamente l'apertura desiderata. È inoltre prevista la gestione dell'indisponibilità delle misure e delle comunicazioni.

Il prototipo utilizza un ESP32 per il TMS, un Arduino UNO per il WCS, un backend Java eseguito su PC per il CUS e un'interfaccia web per il DBS. Le comunicazioni con il CUS avvengono rispettivamente tramite MQTT, connessione seriale e HTTP.

Una breve dimostrazione del prototipo è disponibile nel video dimostrativo.

# 2 Architettura generale del sistema

## 2.1 Componenti e confini di responsabilità

Il sistema adotta un'architettura distribuita con controllo centralizzato nel CUS, responsabile della gestione dello stato globale, delle modalità operative e della determinazione dell'apertura della valvola. Le responsabilità dei diversi sottosistemi sono illustrate nella Figura 1.

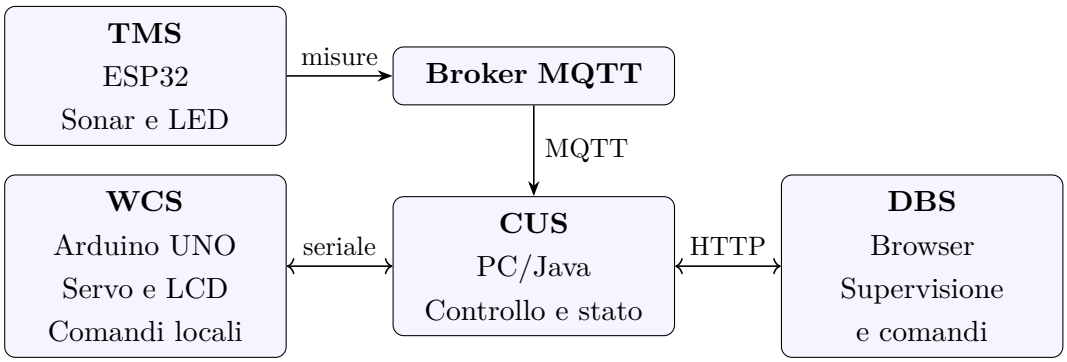



**Figura 1:** Architettura del sistema e collegamenti tra i sottosistemi.

Il TMS acquisisce il livello dell'acqua tramite un sensore sonar e trasmette le misure al CUS. Quest'ultimo elabora i dati ricevuti, applica la politica di controllo e rende disponibile lo stato del sistema. Il WCS gestisce l'attuatore e l'interfaccia locale, consentendo all'operatore di richiedere il cambio di modalità tramite pulsante e la regolazione dell'apertura tramite potenziometro. Il DBS permette il monitoraggio e il controllo remoto attraverso un'interfaccia web.

## 2.2 Comunicazione e propagazione dello stato

La comunicazione tra TMS e CUS avviene tramite MQTT, utilizzando un broker per lo scambio delle misure. Il WCS comunica con il CUS attraverso una connessione seriale, inviando le richieste dell'operatore e ricevendo gli aggiornamenti di stato necessari al controllo dell'attuatore e del display.

Il DBS utilizza HTTP per il monitoraggio remoto e l'invio dei comandi dell'operatore.

I dettagli dei protocolli e dei messaggi sono descritti nella Sezione 7.1.

## 2.3 Stato globale e disponibilità

Il CUS gestisce le modalità operative `AUTOMATIC` e `MANUAL` e monitora la disponibilità delle misure provenienti dal TMS. In assenza di misure valide per un intervallo di tempo prestabilito, il sistema passa allo stato `UNCONNECTED`.

Il DBS distingue questa condizione dalla propria indisponibilità di comunicazione con il CUS, rappresentata come `NOT AVAILABLE`. Il backend può quindi essere raggiungibile anche quando le misure del TMS non sono disponibili.

## 2.4 Scelte architetturali

Nel backend, la logica di controllo è separata dai servizi di comunicazione. L'accesso allo stato condiviso è sincronizzato per gestire le operazioni concorrenti, mantenendo le operazioni di comunicazione al di fuori delle sezioni critiche.

# 3 Tank Monitoring Subsystem (TMS)

Il Tank Monitoring Subsystem (TMS) acquisisce periodicamente il livello di riempimento della tanica e comunica le misure al CUS tramite MQTT.

## 3.1 Architettura software

Il firmware è organizzato in due task FreeRTOS eseguiti su core distinti. La separazione tra acquisizione e comunicazione permette di proseguire le misurazioni anche durante l'attesa o il recupero della connessione. La Tabella 1 riassume le responsabilità dei due task.

| **Task** | **Responsabilità principale** |
|---|---|
| `WaterLevelTask` | Acquisizione e validazione delle letture, calcolo del livello di riempimento. |
| `CommunicationTask` | Gestione Wi-Fi e MQTT, pubblicazione dei dati e aggiornamento dei LED di stato. |

**Tabella 1:** Task principali del firmware TMS.

Lo scambio dei dati avviene tramite una coda condivisa che contiene una sola misura. Ogni nuova lettura sostituisce quella eventualmente non ancora trasmessa, così che al ripristino della comunicazione sia disponibile il dato più recente, senza accumulare uno storico da inviare.

I collegamenti hardware e i parametri operativi sono centralizzati nel file `config.h`. Eventuali errori nella creazione delle risorse RTOS vengono rilevati e segnalati durante l'avvio.

## 3.2 Circuito e componenti hardware

Il prototipo comprende una scheda ESP32, un sonar HC-SR04 e due LED, verde e rosso, per la segnalazione dello stato della comunicazione. I collegamenti sono realizzati su breadboard, come mostrato nella Figura 2.

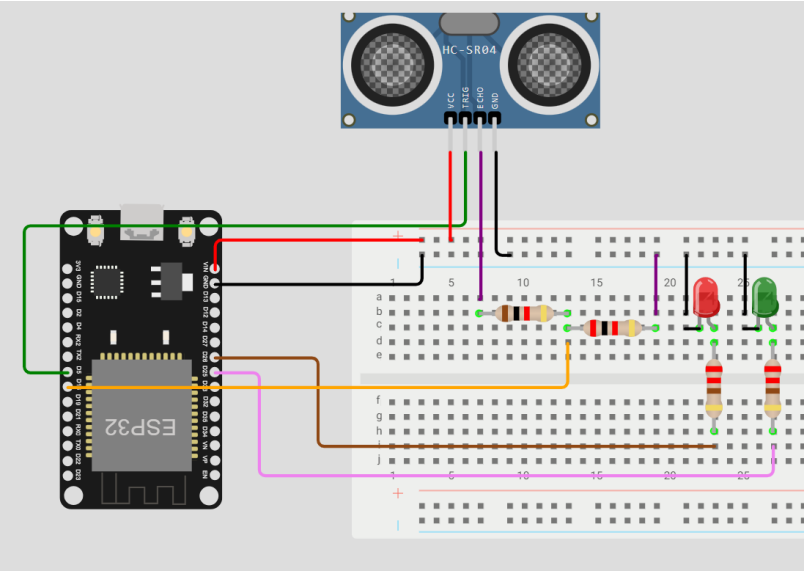



**Figura 2:** Cablaggio del TMS: ESP32, sonar HC-SR04 e LED di stato.

La Tabella 2 riassume l'assegnazione dei segnali.

| GPIO | Collegamento |
|---|---|
| 5 | Segnale TRIG del sonar |
| 18 | Segnale ECHO del sonar, attraverso il partitore resistivo |
| 25 | LED verde |
| 26 | LED rosso |

**Tabella 2:** Assegnazione dei GPIO del TMS.

Il segnale ECHO è adattato agli ingressi a 3,3 V dell'ESP32 mediante un partitore resistivo da 1 kΩ e 2 kΩ. Ogni LED utilizza una resistenza in serie da 220 Ω per limitare la corrente.

## 3.3 Task del firmware

### 3.3.1 CommunicationTask

La logica di comunicazione è descritta da una macchina a stati finiti che distingue le fasi di connessione dalla condizione operativa.

Gli stati principali sono:

- `WIFI_CONNECTING`: attesa della connessione alla rete Wi-Fi
- `MQTT_CONNECTING`: connessione al broker MQTT
- `WAITING_FIRST_PUBLISH`: connessione MQTT stabilita, in attesa della prima pubblicazione riuscita
- `COMMUNICATION_OK`: comunicazione operativa dopo la prima pubblicazione riuscita

La Figura 3 mostra la macchina a stati del task.

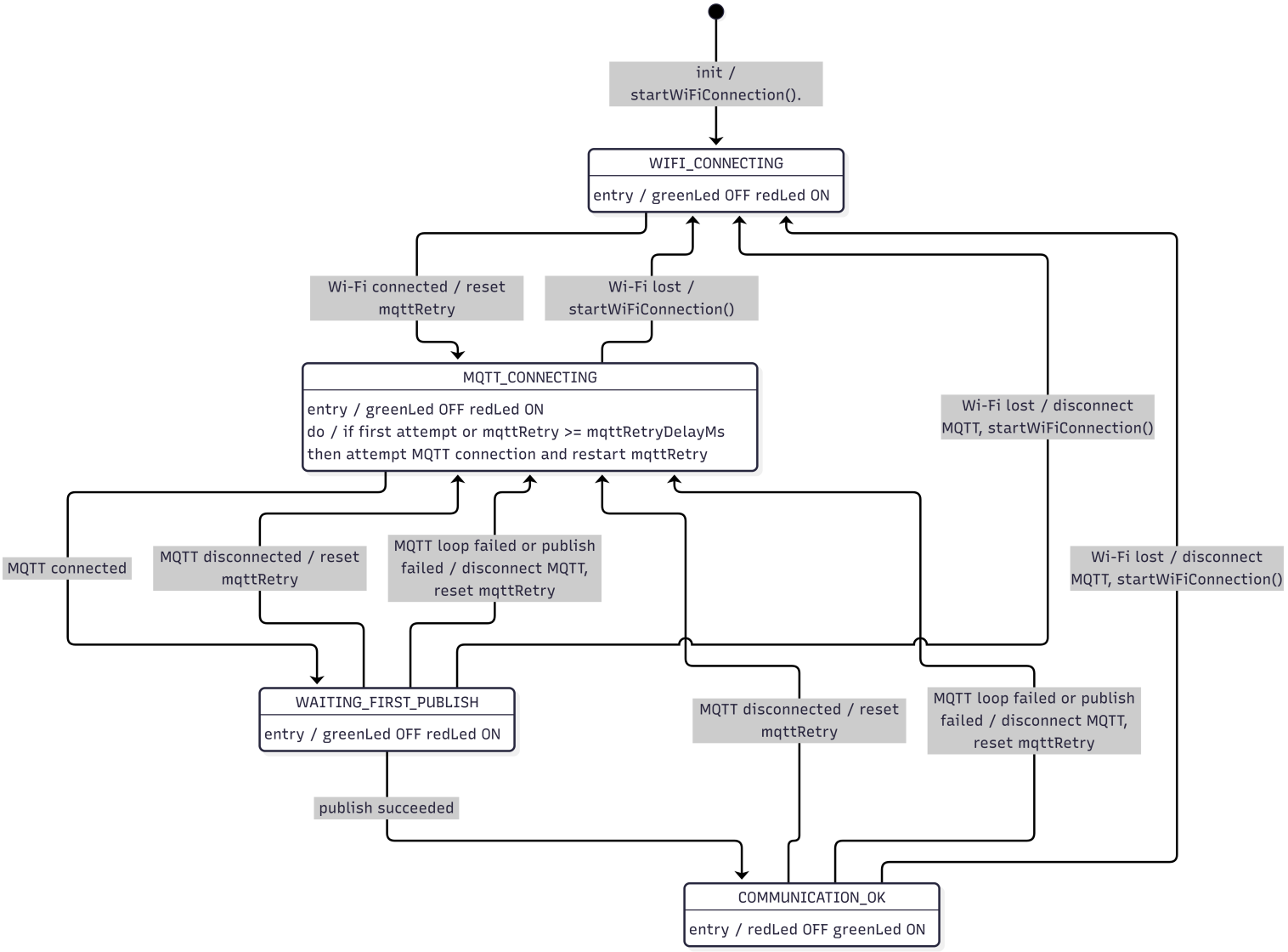



**Figura 3:** FSM del CommunicationTask.

Il task pubblica l'ultima lettura disponibile e mantiene attiva la comunicazione. La perdita del Wi-Fi riporta alla fase di connessione alla rete. Se il Wi-Fi è disponibile, un'interruzione MQTT o un errore di pubblicazione attiva la riconnessione al broker. I tentativi successivi sono separati da un intervallo configurabile.

Il LED verde è acceso e quello rosso è spento nella condizione operativa; negli altri stati la segnalazione è invertita. Dopo una disconnessione, il verde viene riacceso soltanto a seguito di una nuova pubblicazione riuscita. I LED rappresentano lo stato della comunicazione, indipendentemente dalla validità della lettura trasmessa.

Il CUS riceve i messaggi JSON sottoscrivendo lo stesso topic utilizzato dal TMS. Per una misura valida vengono trasmessi il livello in metri e la percentuale di riempimento:

```
{"status":"OK","levelM":0.150,"percentage":50.0}
```

Una lettura non valida viene segnalata mediante:

```
{"status":"NO_ECHO"}
```

### 3.3.2 WaterLevelTask

Il task acquisisce periodicamente la distanza dalla superficie dell'acqua e calcola il livello di riempimento. Il periodo di campionamento è configurabile.

Nel modello adottato, il piano del sensore rappresenta il riempimento al 100%. Indicando con $H$ la distanza tra il sensore e il fondo della tanica e con $d$ la distanza dalla superficie dell'acqua, il livello $h$ e la percentuale $p$ sono calcolati come:

$$h = H - d, \qquad p = \frac{h}{H} \cdot 100$$

La lettura è considerata valida se l'eco viene rilevato e la distanza soddisfa $0 \leq d \leq H$. Il livello risultante è quindi compreso tra zero e $H$, mentre la percentuale appartiene all'intervallo tra 0% e 100%.

Le letture senza eco o fuori intervallo vengono classificate come non valide e il livello non viene calcolato. Anche questo esito viene reso disponibile al task di comunicazione, distinguendolo dalla condizione di tanica vuota, che corrisponde invece a una lettura valida con $d = H$.

## 3.4 Test, criticità e taratura

**Allestimento e verifica delle misure.** Le prove sono state effettuate spostando una scatolina in polimero di forma geometrica regolare davanti al sonar, simulando diversi livelli di riempimento. L'altezza utile è stata configurata a 0,30 m, corrispondenti alla lunghezza del righello utilizzato come riferimento; il periodo di acquisizione è stato impostato a 2 s.

Le distanze rilevate sono risultate approssimativamente corrispondenti a quelle misurate con il righello, fornendo una verifica qualitativa dell'acquisizione. È stata inoltre verificata la coerenza tra livello espresso in metri e percentuale di riempimento.

**Gestione delle letture non valide.** Durante le prove preliminari con la mano come bersaglio sono state osservate letture `NO_ECHO`; è stato quindi utilizzato un oggetto di forma regolare per proseguire le verifiche.

La classificazione delle letture fuori intervallo è stata controllata posizionando il bersaglio oltre i 30 cm. Le letture sono state segnalate come non valide. Il CUS ha riconosciuto i messaggi `NO_ECHO` e restituito `null` per la misura non disponibile, distinguendola da un livello valido pari a zero.

**Avvio e recupero della comunicazione.** All'avvio e dopo un riavvio mediante il pulsante RESET è stata osservata l'accensione iniziale del LED rosso, seguita dal passaggio al verde al ripristino della comunicazione.

È stato inoltre verificato il recupero dopo un'interruzione del Wi-Fi mantenendo la scheda accesa. Al ripristino della rete, il TMS ha ristabilito automaticamente la comunicazione ed è tornato alla segnalazione verde, senza richiedere un riavvio manuale.

# 4 Control Unit Subsystem (CUS)

Il CUS costituisce il nodo centrale del sistema: riceve le misure dal TMS, gestisce le richieste degli operatori e determina l'apertura da comandare alla valvola.

## 4.1 Architettura software

Il CUS è sviluppato in Java 17 e utilizza Maven per la gestione del progetto. Il backend si basa su Vert.x per i servizi HTTP e i timer, Eclipse Paho per MQTT e jSerialComm per la comunicazione seriale.

La logica di controllo è concentrata in `SystemController`, che gestisce lo stato condiviso e coordina `AutomaticPolicy` e `MeasurementHistory`, responsabili rispettivamente della politica automatica e dello storico delle misure. I servizi MQTT, HTTP e seriale gestiscono le comunicazioni con gli altri sottosistemi, come illustrato nella Figura 4.

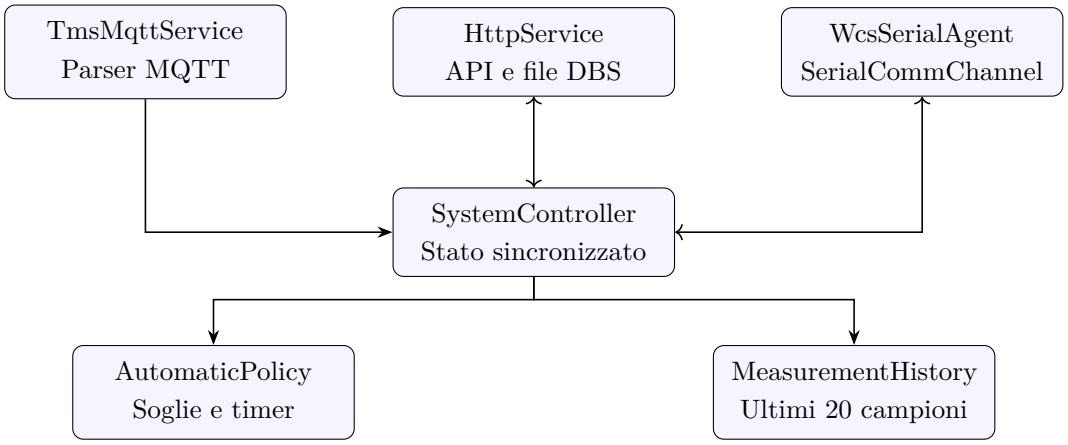



**Figura 4:** Organizzazione software del CUS e relazioni tra i componenti principali.

La configurazione viene letta dal file `cus.properties`, situato nella directory di avvio dell'applicazione. In assenza del file vengono utilizzati i valori predefiniti. Il percorso della directory DBS è relativo alla directory di esecuzione.

I principali parametri del CUS sono riportati nella Tabella 3. Le soglie, gli intervalli temporali T1 e T2 e la porta HTTP sono configurabili; il tick periodico e la capacità dello storico sono invece definiti nel codice.

| Parametro | Valore | Descrizione |
|---|---|---|
| $L_1, L_2$ | $40\%, 70\%$ | Soglie del livello dell'acqua. |
| $T_1$ | 5 s | Intervallo di attesa nella fascia intermedia. |
| $T_2$ | 10 s | Tempo massimo senza misure valide. |
| Tick | 100 ms | Intervallo di controllo periodico. |
| $N$ | 20 | Capacità dello storico delle misure. |
| HTTP | 8080 | Porta del servizio HTTP. |

**Tabella 3:** Parametri principali del CUS e relativi valori predefiniti o definiti nel codice.

## 4.2 Gestione dello stato e politica automatica

All'avvio il sistema si trova in modalità `AUTOMATIC`, con apertura della valvola allo 0% e livello dell'acqua non ancora disponibile. Il conteggio di T2 inizia all'avvio, in attesa della prima misura valida.

### 4.2.1 Politica a stati

La politica automatica è organizzata attraverso una FSM con cinque stati interni: `UNKNOWN`, stato iniziale o successivo a un reset; `LOW`, associato al livello basso; `MID_WAITING`, che rappresenta l'attesa T1; `MID_ACTIVE`, con apertura al 50%; e `HIGH`, con apertura al 100%.

Le condizioni applicate alle misure valide sono riassunte nella Tabella 4.

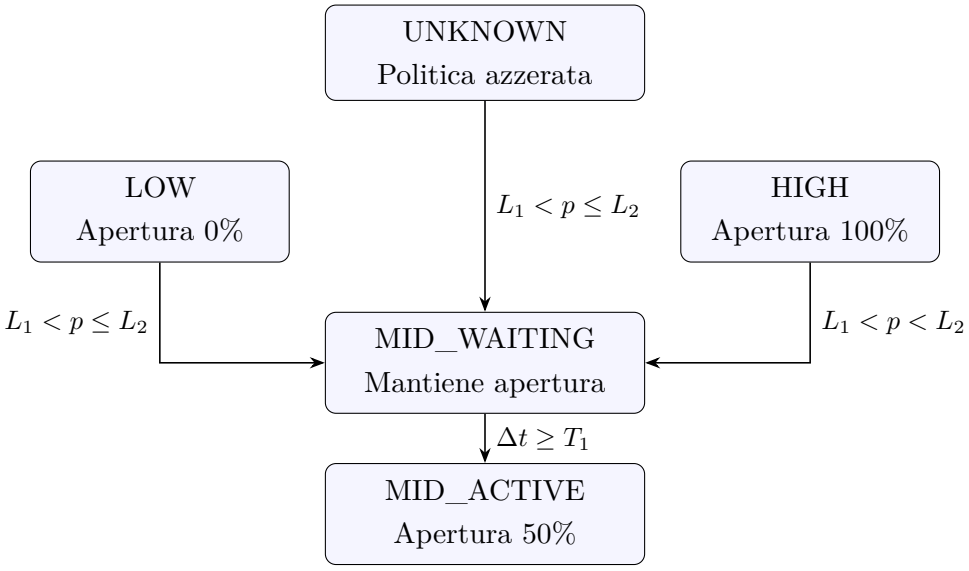
| Livello | Comportamento |
|---|---|
| $p < L_1$ | Passa a `LOW` e imposta l'apertura allo 0%. |
| $p = L_1$ | Mantiene il 50% se si trova già in `MID_ACTIVE`; altrimenti passa a `LOW` e chiude la valvola. |
| $L_1 < p < L_2$ | Mantiene il 50% se già in `MID_ACTIVE`; altrimenti entra in `MID_WAITING`, se necessario, conservando l'apertura precedente durante T1. |
| $p = L_2$ | Mantiene il 100% se già in `HIGH`; altrimenti segue la gestione della fascia intermedia. |
| $p > L_2$ | Passa a `HIGH` e imposta immediatamente l'apertura al 100%. |

**Tabella 4:** Comportamento della politica automatica in funzione del livello e dello stato precedente.

In `MID_WAITING`, al raggiungimento di T1, la politica passa a `MID_ACTIVE` e imposta l'apertura al 50%. La scadenza viene verificata periodicamente dal CUS, anche in assenza di nuove misure.

Durante l'attesa viene mantenuta l'apertura precedente. Ad esempio, dopo una discesa da `HIGH` nella fascia intermedia, il 100% rimane attivo fino alla scadenza di T1, quando l'apertura passa al 50%.

Le principali transizioni della politica automatica sono rappresentate nella Figura 5.




**Condizioni e transizioni aggiuntive:**

- **Soglia L1:** con $p < L_1$ si passa a LOW. A $p = L_1$, MID_ACTIVE mantiene il 50%, mentre gli altri stati passano a LOW.
- **Soglia L2:** con $p > L_2$ si passa a HIGH, che rimane attivo anche a $p = L_2$.
- **Timer T1:** alla scadenza, MID_WAITING passa a MID_ACTIVE, anche senza una nuova misura.
- **Reset:** riporta la politica allo stato UNKNOWN.

**Figura 5:** Stati e principali transizioni della politica automatica.

### 4.2.2 Misure non valide, timeout e recupero

Quando il TMS segnala una lettura non valida tramite `NO_ECHO`, il CUS registra un campione `null` nello storico e rende il livello corrente non disponibile. In modalità `AUTOMATIC`, la politica viene azzerata, mantenendo invariata l'apertura della valvola.

Il timer T2 viene aggiornato soltanto alla ricezione di misure valide. Se non ne arrivano per almeno T2, il CUS passa allo stato `UNCONNECTED` e imposta l'apertura al 5%. Questo valore è stato scelto con l'intento di consentire un deflusso limitato ed evitare uno svuotamento rapido della vasca.

Alla prima misura valida dopo la scadenza di T2, il CUS ripristina la modalità `AUTOMATIC` e rivaluta il livello. La rivalutazione parte internamente dall'apertura di fallback al 5%, senza necessariamente trasmettere al WCS un comando intermedio a tale valore.

### 4.2.3 Comandi manuali e storico

Il passaggio alla modalità `MANUAL` mantiene l'apertura corrente della valvola. L'operatore può impostare un'apertura compresa tra 0% e 100%, tramite l'interfaccia locale o la dashboard. Le misure del TMS continuano ad aggiornare il livello e lo storico, senza modificare l'apertura comandata.

Il ritorno alla modalità `AUTOMATIC` azzera la politica di controllo e rivaluta l'ultima misura, se disponibile. Durante lo stato `UNCONNECTED`, le richieste di cambio modalità e di regolazione dell'apertura vengono rifiutate.

Lo storico conserva in memoria gli ultimi $N$ campioni ricevuti e accettati, comprese le misure non valide rappresentate da `null`. I campioni sono ordinati cronologicamente in base alla ricezione da parte del CUS. Lo storico non è persistente e viene azzerato al riavvio.

## 4.3 Comunicazione e sincronizzazione

Il CUS gestisce eventi provenienti da MQTT, HTTP, comunicazione seriale e timer periodici, che possono essere elaborati concorrentemente. L'accesso allo stato condiviso, alla politica automatica e allo storico è protetto da un lock, mentre le operazioni di comunicazione vengono eseguite al di fuori delle sezioni critiche.

Lo stato del sistema è rappresentato attraverso snapshot immutabili, utilizzati per fornire informazioni coerenti ai componenti esterni.

Gli aggiornamenti destinati al WCS vengono accorpati per privilegiare lo stato più recente, evitando l'accumulo di stati intermedi. Le richieste ricevute tramite seriale vengono invece elaborate dal controller, che ne verifica la compatibilità con lo stato corrente e restituisce lo stato aggiornato.

## 4.4 Validazione e limitazioni

La politica automatica, la gestione dello stato e l'interpretazione dei messaggi MQTT sono state verificate attraverso test unitari JUnit.

Il CUS non dispone di un feedback sulla posizione effettiva della valvola. La comunicazione seriale non prevede un recupero automatico delle interruzioni e i servizi HTTP e MQTT non implementano autenticazione o cifratura.

# 5 Water Channel Subsystem (WCS)

Il WCS gestisce l'attuatore e l'interfaccia operatore locale. Riceve dal CUS lo stato del sistema e il comando di apertura della valvola, aggiornando di conseguenza il servomotore e il display LCD. Attraverso pulsante e potenziometro consente inoltre all'operatore di inviare richieste al CUS, senza modificare autonomamente lo stato globale.

## 5.1 Architettura e hardware

Il WCS è implementato su Arduino UNO con firmware C++ basato sul framework Arduino. La classe `HWPlatform` gestisce l'accesso ai dispositivi hardware, mentre `Context` mantiene l'ultimo stato ricevuto dal CUS, comprendente modalità operativa, apertura comandata e disponibilità dello stato.

I collegamenti dei componenti sono riportati nella Tabella 5 e rappresentati nello schema della Figura 6.

| Pin | Componente e funzione |
|---|---|
| D8 | Pulsante con circuito RC per attenuare i rimbalzi del segnale. |
| A0 | Ingresso analogico del potenziometro. |
| D9 | Segnale di controllo del servomotore. |
| A4 / A5 | Collegamenti SDA / SCL del display LCD I2C. |
| USB | Comunicazione seriale con il CUS a 115200 baud. |

**Tabella 5:** Collegamenti hardware del WCS.

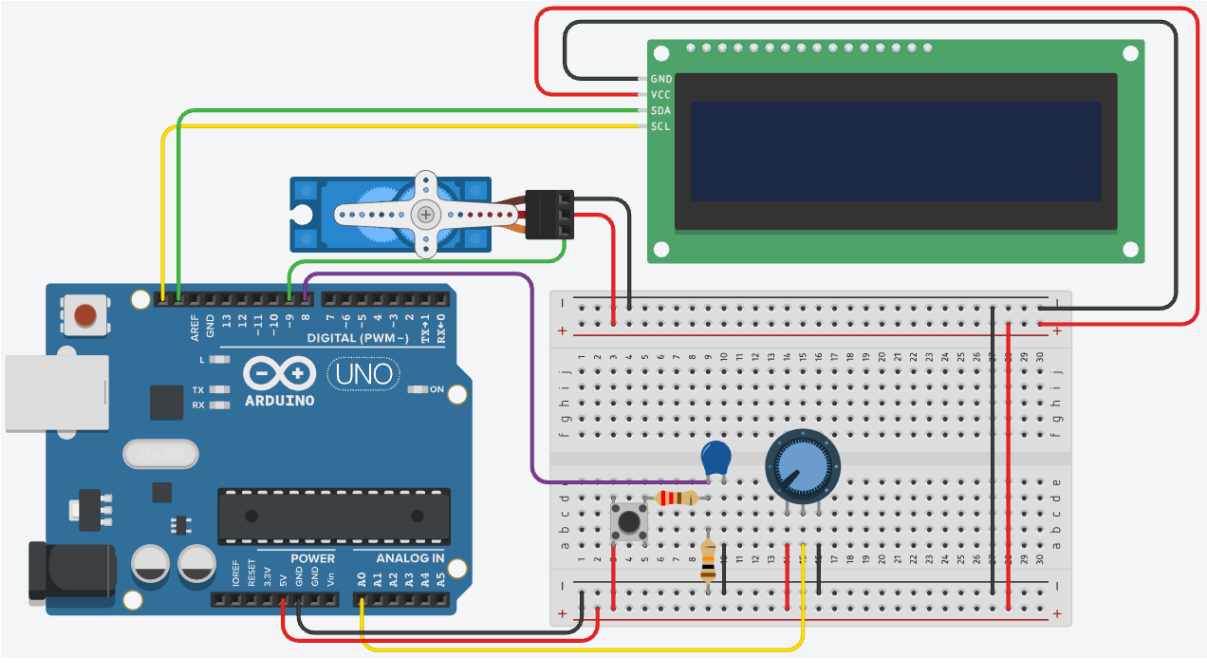



**Figura 6:** Schema dei collegamenti del prototipo WCS realizzato con Tinkercad.

## 5.2 Scheduler, task e macchine a stati

Il firmware è organizzato in task, ciascuno dedicato a una specifica funzione del WCS. La loro esecuzione è gestita da uno scheduler cooperativo, che li richiama in sequenza secondo i rispettivi periodi. Timer1 fornisce il riferimento temporale con un intervallo di base di 50 ms.

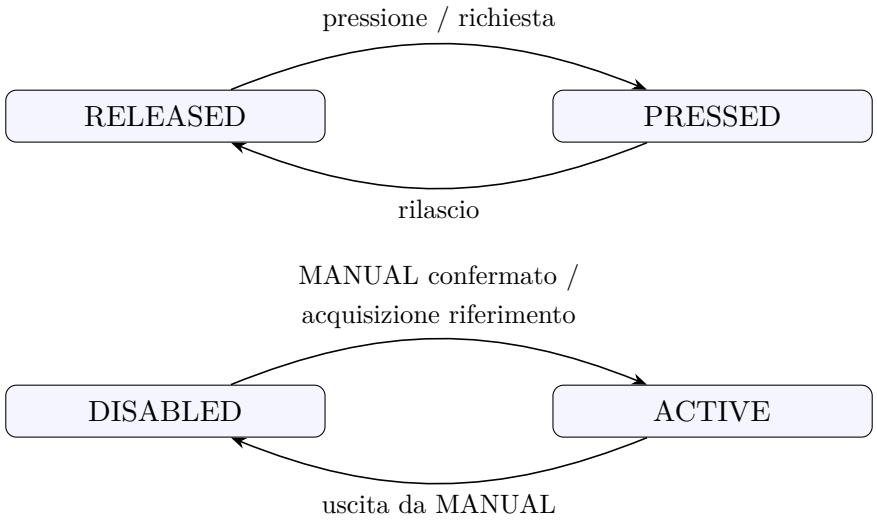
I task, le rispettive responsabilità e i periodi di esecuzione sono riportati nella Tabella 6.

| Task | Periodo | Responsabilità |
|---|---|---|
| SerialCommTask | 50 ms | Gestione della comunicazione seriale con il CUS. |
| ButtonTask | 50 ms | Rilevamento delle pressioni e richieste di cambio modalità. |
| PotentiometerTask | 100 ms | Lettura del potenziometro e richieste di apertura. |
| ValveTask | 50 ms | Aggiornamento del servomotore. |
| DisplayTask | 200 ms | Aggiornamento delle informazioni sul display LCD. |

**Tabella 6:** Task del WCS e relativi periodi di esecuzione.

I task condividono il `Context`, che contiene lo stato ricevuto dal CUS. Timer1 è dedicato allo scheduler, mentre Timer2 viene utilizzato dalla libreria `ServoTimer2` per il controllo del servomotore.

La gestione del pulsante e del potenziometro è organizzata attraverso due FSM, rappresentate nella Figura 7.




**Condizioni operative:**

- **Pulsante:** una richiesta per pressione, senza ripetizioni fino al rilascio.
- **Potenziometro:** in ACTIVE invia una richiesta per variazioni di almeno 5 punti percentuali o al raggiungimento di un estremo con variazione non nulla.
- **Stati iniziali:** RELEASED e DISABLED.

**Figura 7:** Macchine a stati per la gestione del pulsante e del potenziometro.

`ButtonTask` distingue gli stati `RELEASED` e `PRESSED`, generando una richiesta al rilevamento della pressione. Il circuito RC attenua i rimbalzi del pulsante, mentre la FSM evita richieste

ripetute finché il segnale rimane attivo. Prima della sincronizzazione con il CUS e durante `UNCONNECTED`, i comandi locali sono disabilitati.

`PotentiometerTask` opera invece negli stati `DISABLED` e `ACTIVE`. Quando il CUS conferma l'ingresso in `MANUAL`, il task acquisisce la posizione corrente del potenziometro come riferimento, senza inviare immediatamente un comando.

Le successive variazioni della manopola generano richieste di apertura quando raggiungono una soglia minima di spostamento. Sono inoltre gestiti gli estremi di apertura, per consentire il raggiungimento completo dello 0% e del 100%.

## 5.3 Interfaccia operatore e controllo della valvola

All'avvio il WCS mantiene il servomotore in posizione di chiusura e il display mostra un messaggio di attesa. Prima di rendere operativi i controlli locali, il sistema si sincronizza con il CUS e acquisisce uno stato valido.

Il pulsante permette di richiedere il cambio tra le modalità `AUTOMATIC` e `MANUAL`. In modalità `MANUAL`, il potenziometro consente di richiedere una diversa apertura della valvola.

Il WCS aggiorna servomotore e display soltanto in base allo stato ricevuto dal CUS.

Il task `ValveTask` converte la percentuale di apertura comandata, compresa tra 0% e 100%, in un angolo di comando del servomotore tra 0° e 90°.

Il display LCD mostra la modalità operativa e la percentuale di apertura della valvola. Le informazioni vengono aggiornate quando cambia lo stato ricevuto.

## 5.4 Validazione e limitazioni

Le prove funzionali sul prototipo hanno confermato il comportamento dei controlli locali, dell'attuatore, del display e della comunicazione con il CUS.

Il WCS non dispone di un feedback sulla posizione effettiva del servomotore e non gestisce automaticamente il ripristino delle interruzioni della comunicazione seriale.

# 6 Dashboard Subsystem (DBS)

Il DBS è l'interfaccia web per il monitoraggio del livello dell'acqua, della modalità operativa, dell'apertura comandata e dello storico, e per l'invio di richieste al CUS.

## 6.1 Architettura dell'interfaccia web

L'interfaccia è sviluppata con HTML, CSS e JavaScript, senza framework frontend. Comprende pannelli informativi, controlli per l'operatore e un grafico per la visualizzazione dello storico. Moduli distinti gestiscono la comunicazione HTTP, l'interfaccia e il grafico dello storico, realizzato con Chart.js incluso localmente.

## 6.2 Comunicazione HTTP e gestione dello stato

Il DBS comunica con il CUS tramite HTTP, recuperando periodicamente lo stato del sistema e lo storico delle misure. Nella configurazione attuale, le richieste sono indirizzate a `http://localhost:8080`, con intervalli di aggiornamento nominali di 1000 ms per entrambi i dati.

L'interfaccia distingue le misure non disponibili, rappresentate da `null`, da un livello valido pari allo 0%. Nel grafico, le misure non valide sono rappresentate mediante interruzioni della linea.

La dashboard distingue inoltre due condizioni di indisponibilità. Lo stato `UNCONNECTED` viene comunicato dal CUS quando non riceve misure valide dal TMS per un intervallo prolungato. La condizione `NOT AVAILABLE` indica invece che il DBS non riesce a ottenere uno stato utilizzabile dal CUS.

In quest'ultimo caso, i comandi vengono disabilitati e gli ultimi valori disponibili rimangono visualizzati, senza essere considerati informazioni aggiornate. Un errore nel recupero del solo storico viene invece segnalato separatamente, senza modificare lo stato di disponibilità del CUS.

## 6.3 Interazione dell'operatore

L'interfaccia mette a disposizione un pulsante per richiedere il cambio tra le modalità `AUTOMATIC` e `MANUAL` e uno slider per regolare l'apertura della valvola.

Lo slider è abilitato soltanto in modalità `MANUAL`, quando il CUS è disponibile. Durante la regolazione viene mostrato il valore selezionato, mentre la richiesta di apertura viene inviata al termine della modifica.

La dashboard aggiorna i valori del sistema sulla base dello stato restituito dal backend, senza considerare automaticamente applicato un comando appena inviato.

Durante l'elaborazione delle richieste, i relativi controlli vengono temporaneamente disabilitati.

Le prove funzionali hanno verificato l'interfaccia anche durante l'indisponibilità e il ripristino del CUS. Il DBS non verifica direttamente la disponibilità del WCS.

# 7 Integrazione e comunicazione

## 7.1 Interazione tra sottosistemi e contratti

I sottosistemi comunicano attraverso tre protocolli: MQTT tra TMS e CUS, seriale tra WCS e CUS e HTTP tra DBS e CUS.

Nel funzionamento automatico, il TMS invia le misure attraverso il broker MQTT. Il CUS le elabora, aggiorna lo stato del sistema e, quando necessario, trasmette al WCS il nuovo comando di apertura. Il WCS aggiorna di conseguenza il servomotore e il display LCD.

Il DBS consulta il CUS tramite HTTP indipendentemente dagli aggiornamenti seriali al WCS. La Figura 8 rappresenta il flusso delle informazioni tra i sottosistemi.

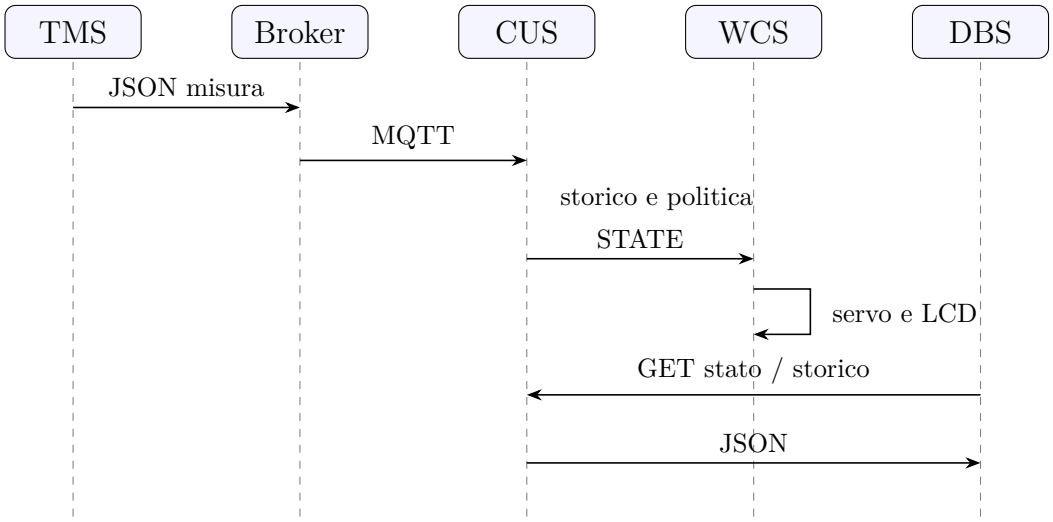



**Figura 8:** Sequenza logica di una misura che determina un aggiornamento del sistema

**MQTT.** Il TMS pubblica le misure su un topic MQTT condiviso con il CUS, utilizzando messaggi JSON. Una misura valida contiene lo stato `OK`, il livello dell'acqua in metri e la percentuale di riempimento. Una lettura non valida viene invece segnalata attraverso `NO_ECHO`.

Il CUS utilizza la percentuale ricevuta per la politica automatica e assegna il timestamp ai campioni quando vengono ricevuti. La comunicazione utilizza QoS 0 e non prevede una conferma applicativa dell'elaborazione della misura.

**Seriale.** La comunicazione tra WCS e CUS utilizza un protocollo testuale a 115200 baud. Il WCS invia richieste di sincronizzazione e comandi dell'operatore, mentre il CUS trasmette gli aggiornamenti dello stato.

I principali messaggi sono riportati nella Tabella 7.

| Messaggio | Significato |
|---|---|
| `READY` | Richiesta di sincronizzazione dal WCS |
| `CMD,MODE,...` | Richiesta di cambio modalità dal WCS |
| `CMD,VALVE,...` | Richiesta di apertura dal WCS |
| `STATE,...` | Stato corrente trasmesso dal CUS al WCS |
| `LOG,...` | Messaggio diagnostico opzionale del WCS |

**Tabella 7:** Principali messaggi del protocollo seriale tra WCS e CUS

Il messaggio `STATE` rappresenta quindi lo stato definito dal controller, non una conferma dell'effettiva posizione del servomotore.

**HTTP.** Le operazioni HTTP disponibili al DBS sono riportate nella Tabella 8.

| Operazione | Funzione |
|---|---|
| `GET /api/state` | Restituisce modalità, apertura e livello corrente. |
| `GET /api/measurements` | Restituisce gli ultimi $N$ campioni dello storico. |
| `POST /api/mode` | Richiede il cambio di modalità operativa. |
| `POST /api/valve` | Richiede una nuova apertura in modalità MANUAL. |

**Tabella 8:** Operazioni HTTP esposte dal CUS e utilizzate dal DBS

## 7.2 Scenari operativi e gestione delle interruzioni

**Avvio e sincronizzazione.** All'avvio il CUS inizializza il sistema in modalità `AUTOMATIC`, mentre il WCS mantiene la valvola chiusa fino alla ricezione di uno stato valido. La sincronizzazione avviene attraverso i messaggi `READY` e `STATE`.

Il DBS acquisisce lo stato iniziale dal CUS attraverso il collegamento HTTP.

**Controllo locale e remoto.** Il WCS e il DBS permettono entrambi di richiedere il cambio di modalità e la regolazione dell'apertura. Le richieste vengono valutate dal CUS in base allo stato corrente, senza una priorità prestabilita tra le due interfacce.

**Indisponibilità delle misure.** La gestione delle letture non valide e del timeout T2 è descritta nella Sezione 4.2.2. Il passaggio a `UNCONNECTED` viene comunicato al WCS insieme all'apertura di fallback e reso disponibile al DBS, che segnala anche l'indisponibilità del livello. Il ripristino delle misure si riflette nei successivi aggiornamenti di entrambe le interfacce.

**Interruzioni delle comunicazioni.** Se il DBS non riesce a comunicare con il CUS, mostra `NOT AVAILABLE`, senza modificare lo stato degli altri sottosistemi.

Un'interruzione della comunicazione seriale non impedisce al CUS di elaborare le misure del TMS e le richieste HTTP, ma il WCS può mantenere l'ultimo comando ricevuto.

L'indisponibilità del TMS non implica quindi necessariamente quella del CUS o del DBS.

## 7.3 Validazione del sistema integrato

Il prototipo completo è stato sottoposto a prove funzionali con i quattro sottosistemi collegati e operativi. Il video dimostrativo richiamato nell'introduzione documenta il funzionamento congiunto del sistema negli scenari mostrati.

# 8 Conclusioni

## 8.1 Risultati e limitazioni

Il progetto ha portato alla realizzazione di un sistema distribuito per il monitoraggio del livello dell'acqua e il controllo dell'apertura di una valvola. L'architettura adottata affida al CUS la gestione centralizzata dello stato e della politica di controllo, integrando acquisizione delle misure, attuazione e interfacce operatore locali e remote.

Le principali limitazioni riguardano l'assenza di un feedback sulla posizione effettiva della valvola e il rilevamento delle interruzioni della comunicazione seriale.

## 8.2 Possibili sviluppi futuri

Tra i possibili miglioramenti rientrano l'introduzione di un feedback sulla posizione della valvola, il monitoraggio della comunicazione tra CUS e WCS, la persistenza dello storico delle misure e l'autenticazione delle richieste di controllo.

Queste estensioni potrebbero migliorare l'affidabilità e la gestione del sistema, pur non rientrando negli obiettivi dell'attuale implementazione.

# Riferimenti bibliografici

[1] A. Ricci, *Assignment #03 – Smart Tank Monitoring System*, Embedded Systems and Internet of Things, a.a. 2025/2026, versione 1.0.0-20251112. Consegna ufficiale nel repository del corso. Consultato il 3 novembre 2026.